



INFORME DE GUÍA PRÁCTICA

I. PORTADA

Tema:	APE3_Arboles
Unidad de Organización Curricular:	BÁSICA
Nivel y Paralelo:	Tercero - B
Alumnos participantes:	Tituaña Calapiña Diana Pamela
Asignatura:	Estructura de Datos
Docente:	Ing. Jose Caiza, Mg.

II. INFORME DE GUÍA PRÁCTICA

2.1 Objetivos

General:

Desarrollar habilidades en el trabajo de árboles.

Específicos:

- Implementar la lógica de inserción en un Árbol Binario de Búsqueda (BST) manteniendo sus propiedades.
- Calcular la profundidad máxima (altura) de árboles binarios mediante recursión.
- Generar listados ordenados mediante el recorrido In-Order en BST.
- Transformar árboles binarios mediante inversión (intercambio de hijos izquierdo y derecho).
- Comparar las implementaciones en C++ y Java identificando diferencias sintácticas y de gestión de memoria.

2.2 Modalidad

Presencial.

2.3 Tiempo de duración

Presenciales: 8

No presenciales: 0

2.4 Instrucciones

Lleve a cabo las actividades indicadas.

Suba a la plataforma un informe con el resultado obtenido

2.5 Listado de equipos, materiales y recursos

Listado de equipos y materiales generales empleados en la guía práctica:

- Inteligencia artificial
- TAC StarUML
- Apuntes de clase
- Bibliografía virtual

TAC (Tecnologías para el Aprendizaje y Conocimiento) empleados en la guía práctica:

- ☒ Plataformas educativas
- ☐ Simuladores y laboratorios virtuales
- ☐ Aplicaciones educativas
- ☒ Recursos audiovisuales
- ☐ Gamificación
- ☒ Inteligencia Artificial

Otros (Especifique): _____

2.6 Actividades por desarrollar



Desarrollar lo indicado en archivo adjunto. Luego, elaborar un informe del trabajo llevado a cabo donde resalte los aspectos fundamentales tenidos en cuenta en su propuesta de solución.

2.7 Resultados obtenidos

1. Introducción

Este informe presenta la implementación y demostración de cinco ejercicios sobre estructuras de árboles, desarrollados en C++ y Java. Los ejercicios abarcan operaciones fundamentales sobre árboles N-arios y árboles binarios de búsqueda: conteo de nodos, inserción, cálculo de altura, recorrido InOrder y transformación (inversión) del árbol. Cada ejercicio incluye el código implementado, la estructura del árbol utilizada, y la verificación de los resultados esperados contra los obtenidos.

2. Objetivos

Objetivo General

Implementar y analizar, mediante un menú interactivo desarrollado en C++ y Java, las operaciones fundamentales de transformación, reducción y presentación sobre árboles binarios, con el propósito de comprender su comportamiento recursivo y su aplicación práctica en estructuras de datos no lineales.

Objetivos específicos

- Implementar la estructura base de un árbol binario (nodo con valor, punteros/enlaces a subárboles izquierdo y derecho) en ambos lenguajes, asegurando la correcta creación y manipulación de nodos.
- Desarrollar operaciones de transformación y reducción, incluyendo el espejo (inversión del árbol), la conversión a suma de hijos, y el podado de hojas, verificando su efecto sobre la estructura del árbol.
- Aplicar recorridos de presentación (inorden, preorden, postorden y por niveles) para visualizar el estado del árbol antes y después de cada transformación, validando los resultados esperados.

3. Marco Teórico

3.1. Árbol N-ario

Un árbol N-ario generaliza el concepto de árbol binario al permitir que cada nodo tenga un número arbitrario de hijos. Se utiliza comúnmente para representar jerarquías donde el número de ramas no es fijo, como sistemas de archivos, organigramas o menús de aplicaciones. En la implementación, cada nodo almacena sus hijos en una colección dinámica (vector en C++ o List en Java).

3.2. Recursividad en Árboles

La recursividad es la técnica algorítmica más natural para trabajar con árboles, dado que un árbol puede definirse recursivamente: un árbol es un nodo raíz más un conjunto de subárboles (cada uno de ellos también un árbol). Cualquier operación sobre un árbol puede descomponerse en: resolver el caso base (nodo nulo o hoja), y aplicar la misma operación sobre los subárboles izquierdo y derecho. Este patrón, conocido como División y Conquista, es la base de los cinco ejercicios implementados en este informe.

3.3. Principales Operaciones sobre Árboles Binarios

Las operaciones fundamentales implementadas en este informe son las siguientes:



- Inserción en BST: ubicar recursivamente la posición correcta comparando el valor con la raíz actual, garantizando la propiedad de orden.
- Conteo de nodos: recorrer todos los nodos del árbol y acumular un contador. La complejidad es $O(n)$ ya que se visita cada nodo exactamente una vez.
- Cálculo de altura: determinar recursivamente la rama más larga del árbol tomando el máximo entre la altura del subárbol izquierdo y el derecho.
- Recorrido InOrder: visitar los nodos en el orden izquierda-raíz-derecha, produciendo los valores en orden ascendente en un BST.
- Inversión (espejo): intercambiar recursivamente el hijo izquierdo y el hijo derecho de cada nodo, transformando el árbol en su imagen especular.

4. Documentación

4.1. Contar Nodos

Se implementa la función contarNodos() sobre un árbol N-ario (cada nodo puede tener múltiples hijos). La función recorre todos los nodos del árbol de forma recursiva, sumando 1 por cada nodo visitado. El caso base retorna 0 cuando el nodo es nulo.

4.1.1. Implementación

C++

```
int contarNodos(NodoN* raiz) {
    if (raiz == nullptr) return 0;
    int total = 1;
    for (NodoN* hijo : raiz->hijos){
        total += contarNodos(hijo);
    }
    return total;
}
```

Java

```
public static int contarNodos(NodoN var0) {
    if (var0 == null) {
        return 0;
    } else {
        int var1 = 1;
        for (NodoN var3 : var0.hijos) {
            var1 += contarNodos(var3);
        }
        return var1;
    }
}
```

Tabla I. Contar Nodos

Prueba	Esperado	Obtenido
Nodos totales	6	6 (CORRECTO)

La función itera sobre los 6 nodos del árbol (1, 2, 3, 4, 5, 6) y retorna el total correcto. La lógica es idéntica en C++ y Java; la única diferencia es el uso de `vector<NodoN*>` en C++ frente a `List<NodoN>` en Java para almacenar los hijos.

4.2. Función insertar

Se implementa la función insertar() para un árbol binario de búsqueda. La función compara cada valor con la raíz actual: si es menor, navega al subárbol izquierdo; si es mayor o igual, al subárbol derecho. El proceso se repite recursivamente hasta encontrar un espacio nulo donde insertar el nuevo nodo.

Árbol Construido

```

    10      <- raíz
   /  \
  5    15   <- insertados
 /
3          <- insertado
```

Implementación



C++

```
Nodo* insertar(Nodo* raiz, int valor) {  
    if (raiz == nullptr) return new Nodo(valor);  
    if (valor < raiz->valor) {  
        raiz->izquierdo = insertar(raiz->izquierdo, valor);  
    } else {  
        raiz->derecho = insertar(raiz->derecho, valor);  
    }  
  
    return raiz;  
}
```

Java

```
public static Nodo insertar(Nodo raiz, int valor) {  
    if (raiz == null) return new Nodo(valor);  
    if (valor < raiz.valor) {  
        raiz.izquierdo = insertar(raiz.izquierdo, valor);  
    } else {  
        raiz.derecho = insertar(raiz.derecho, valor);  
    }  
  
    return raiz;  
}
```

Tabla II. Insertar

Verificación	Esperado	Obtenido
Raíz	10	10
Hijo izquierdo de raíz	5	5
Hijo derecho de raíz	15	15
Hijo izquierdo del 5	3	3

La propiedad BST se mantiene correctamente: todos los valores menores a la raíz (10) quedan en el subárbol izquierdo (5, 3) y los mayores en el derecho (15). La función retorna siempre la raíz del subárbol modificado, permitiendo la asignación recursiva correcta.

4.3. Calcular Altura

Se implementa calcularAltura() que determina el número de niveles del árbol. La altura se define como 1 más el máximo entre la altura del subárbol izquierdo y el derecho. Un árbol nulo tiene altura 0. Se utiliza std::max en C++ y Math.max en Java

Arbol de Prueba

```
1      <- nivel 1  
  \  
  2      <- nivel 2  
  /\  
 3      <- nivel 3 (altura = 3)
```

Implementación

C++

```
int calcularAltura(Nodo* raiz) {  
    if (raiz == nullptr) return 0;  
    return 1 + max(calcularAltura(raiz->izquierdo), calcularAltura(raiz->derecho));  
}
```

Java



```
public static int calcularAltura(Nodo raiz) {  
    if (raiz == null) return 0;  
    return 1 + Math.max(calcularAltura(raiz.izquierdo), calcularAltura(raiz.derecho));  
}
```

Tabla III. Calcular Altura

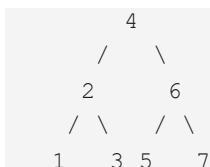
Prueba	Esperado	Obtenido
Altura del árbol	3	3
Altura de árbol nulo	0	0

El árbol de prueba es intencionalmente desbalanceado (solo tiene hijo derecho en la raíz, y solo hijo izquierdo en el nodo 2), lo que permite verificar que la función no asume un árbol completo y calcula correctamente siguiendo la rama más profunda.

4.4. Inorden

Se implementa el recorrido InOrder que visita los nodos en el orden: subárbol izquierdo, raíz, subárbol derecho. En un árbol BST este recorrido produce siempre los valores en orden ascendente. El resultado se acumula en un vector (C++) o List (Java) para facilitar la verificación.

Árbol de Prueba



Implementación

C++

```
void inOrderAux(Nodo* nodo, vector<int>& resultado)  
{  
    if (nodo == nullptr) return;  
    inOrderAux(nodo->izquierdo, resultado);  
    resultado.push_back(nodo->valor);  
    inOrderAux(nodo->derecho, resultado);  
}
```

Java

```
public static void inOrderAux(Nodo nodo, List<Integer> resultado) {  
    if (nodo == null) return;  
    inOrderAux(nodo.izquierdo, resultado);  
    resultado.add(nodo.valor);  
    inOrderAux(nodo.derecho, resultado);  
}
```



Tabla IV. Inorden

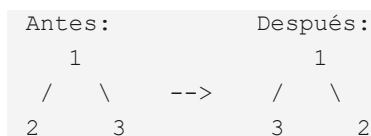
Verificación	Esperado	Obtenido
InOrder completo	[1, 2, 3, 4, 5, 6, 7]	[1, 2, 3, 4, 5, 6, 7]

El recorrido InOrder sobre el árbol BST de prueba produce los 7 elementos en orden ascendente, confirmando tanto la correctitud del algoritmo como la propiedad de orden del BST. La función auxiliar usa un parámetro por referencia (C++) o por referencia de objeto (Java) para acumular los valores sin necesidad de variables globales.

4.5. Invertir

Se implementa la función `invertir()` que transforma un árbol en su imagen especular: intercambia el hijo izquierdo y el hijo derecho de cada nodo de manera recursiva. La función procesa primero los subárboles y luego realiza el intercambio en el nodo actual, garantizando que toda la estructura quede correctamente invertida.

Transformación Esperada



Implementación

C++

```
Nodo* invertir(Nodo* raiz) {
    if (raiz == nullptr) return nullptr;
    // Intercambio
    Nodo* temp = raiz->izquierdo;
    raiz->izquierdo = invertir(raiz->derecho);
    raiz->derecho = invertir(temp);
    return raiz;
}
```

Java

```
public static Nodo invertir(Nodo raiz) {
    if (raiz == null) return null;
    // Intercambio
    Nodo temp = raiz.izquierdo;
    raiz.izquierdo = invertir(raiz.derecho);
    raiz.derecho = invertir(temp);
    return raiz;
}
```

Tabla V. Invertir

Estado	Hijo Izquierdo	Hijo Derecho
Antes de invertir	2	3



Después de invertir	3	2
---------------------	---	---

5. Comparativas entre C++ y Java

Ejercicio	C++	Java
E1: Contar nodos	vector<NodoN*>	List<NodoN>
E2: Insertar BST	Nodo* (puntero)	Nodo (referencia)
E3: Altura	std::max / nullptr	Math.max / null
E4: InOrder	vector<int>& ref.	List<Integer>
E5: Invertir	Swap con temp	Swap con temp

6. Conclusiones

- Los cinco ejercicios producen los resultados esperados en ambos lenguajes, confirmando que la lógica de los algoritmos es correcta e independiente del lenguaje de implementación.
- La recursividad es la herramienta central en todos los ejercicios: el caso base (nodo nulo) garantiza la terminación y la llamada recursiva resuelve el subproblema de forma natural.
- El recorrido InOrder sobre un BST confirma la propiedad de orden: los 7 valores del árbol de prueba aparecen en secuencia ascendente [1, 2, 3, 4, 5, 6, 7].
- La función invertir() demuestra el patrón clásico de intercambio con variable temporal aplicado de forma recursiva, garantizando que toda la estructura quede en espejo.
- La principal diferencia entre C++ y Java es la gestión de memoria: C++ usa punteros explícitos (Nodo*) y requiere verificar nullptr, mientras que Java maneja referencias de forma automática con Garbage Collector.

2.8 Habilidades blandas empleadas en la práctica

- ☐ Liderazgo
- ☐ Trabajo en equipo
- ☐ Comunicación asertiva
- ☐ La empatía
- ☐ Pensamiento crítico
- ☒ Flexibilidad
- ☒ La resolución de conflictos
- ☒ Adaptabilidad
- ☒ Responsabilidad

2.9 Conclusiones



- La implementación de la inserción en un Árbol Binario de Búsqueda (BST) permitió comprender cómo se mantienen las propiedades fundamentales del árbol, garantizando que los valores menores se ubiquen a la izquierda y los mayores a la derecha. Esto facilita búsquedas y operaciones eficientes.
- El cálculo de la profundidad máxima mediante recursión evidenció la importancia de este paradigma para resolver problemas jerárquicos en estructuras de datos, ya que simplifica el recorrido y análisis de los nodos del árbol.
- El recorrido In-Order demostró ser una técnica eficaz para obtener los elementos del BST en orden ascendente, confirmando la utilidad de los árboles binarios de búsqueda para organizar información de forma estructurada.
- La inversión de árboles binarios permitió entender cómo transformar la estructura de un árbol intercambiando sus hijos izquierdo y derecho, reforzando el manejo de punteros o referencias y el uso de algoritmos recursivos.
- La comparación entre C++ y Java mostró diferencias importantes en sintaxis, manejo de memoria y administración de objetos. Mientras C++ ofrece mayor control mediante punteros y gestión manual de memoria, Java simplifica este proceso con el recolector de basura (Garbage Collector), aunque con menor control directo sobre los recursos.
- El desarrollo de estas actividades fortaleció las habilidades de programación orientada a estructuras dinámicas, análisis algorítmico y resolución de problemas utilizando árboles binarios.
- Al integrar los diferentes diagramas, ayuda a la comprensión de cómo se realiza cada actividad desde las diferentes perspectivas. Lo cual ayuda a una mejor comprensión de todo el sistema.

2.10 Recomendaciones

- Practicar constantemente la implementación de árboles binarios y BST para mejorar la comprensión de recorridos, inserciones y eliminaciones de nodos.
- Utilizar diagramas o representaciones gráficas de árboles antes de programar, ya que ayudan a visualizar mejor la estructura y el comportamiento de los algoritmos.
- Reforzar el estudio de la recursividad, debido a que es un concepto fundamental en la manipulación de árboles y otras estructuras jerárquicas.
- Comparar diferentes lenguajes de programación, como C++ y Java, para comprender cómo cada uno gestiona la memoria, los objetos y la sintaxis aplicada a estructuras dinámicas.
- Implementar casos de prueba con distintos tamaños de árboles para verificar el correcto funcionamiento de los algoritmos y detectar posibles errores en profundidad, inserción o recorridos.
- Investigar estructuras más avanzadas derivadas de los árboles binarios, como árboles AVL, árboles Rojo-Negro o heaps, para ampliar el conocimiento en estructuras de datos eficientes.
- Mantener buenas prácticas de programación, incluyendo comentarios, modularización del código y validación de datos, con el fin de facilitar el mantenimiento y comprensión de los programas.

2.11 Referencias bibliográficas

[1] EnjoyAlgorithms, “N-ary Tree Data Structure,” *EnjoyAlgorithms*, 2022. [En línea]. Disponible en: [EnjoyAlgorithms](https://www.enjoyalgorithms.com/blog/n-ary-tree-data-structure). [Accedido: 12-may-2026].

[2] Universidad de Alicante, “Estructuras de Datos y Árboles,” *RUA Universidad de Alicante*. [En línea]. Disponible en: [RUA Universidad de Alicante](https://www.ua.es/informatica/estructuras-de-datos-y-arboles). [Accedido: 12-may-2026].



[3] Universidad de Valencia, “Tema 14: Árboles,” *Departamento de Informática*. [En línea]. Disponible en: [Universidad de Valencia - Tema 14](#). [Accedido: 12-may-2026].

[4] A. Iderepn, “Programación I,” *WordPress*, 2020. [En línea]. Disponible en: [Programación I WordPress](#). [Accedido: 12-may-2026].

2.12 Anexos

Link GitHub: <https://github.com/tituanadiana65-glitch/APE-ARBOLES>

Estructura:

```
APE-ARBOLES/
├── APE3_ARBOLES/
│   ├── .vscode/
│   │   └── settings.json
│   ├── APE_Arboles/
│   │   ├── cpp/
│   │   │   ├── Ejercicio1_Basico.cpp
│   │   │   ├── Ejercicio2_Binario.cpp
│   │   │   ├── Ejercicio3_Binario.cpp
│   │   │   ├── Ejercicio4_Recorridos.cpp
│   │   │   ├── Ejercicio5_Transformacion.cpp
│   │   │   ├── ejercicio1.exe
│   │   │   ├── ejercicio2.exe
│   │   │   ├── ejercicio3.exe
│   │   │   ├── ejercicio4.exe
│   │   │   └── ejercicio5.exe
│   │   └── java/
│   │       ├── Ejercicio1_Basico.java
│   │       ├── Ejercicio1_Basico.class
│   │       ├── Ejercicio2_Binario.java
│   │       ├── Ejercicio2_Binario.class
│   │       ├── Ejercicio3_Binario2.java
│   │       ├── Ejercicio3_Binario2.class
│   │       ├── Ejercicio5_Transformacion.java
│   │       ├── Ejercicio5_Transformacion.class
│   │       ├── RecorridoInOrder.java
│   │       ├── RecorridoInOrder.class
│   │       ├── Nodo.class
│   │       └── NodoN.class
│   ├── README.md
│   ├── INFORME_APE3.pdf
│   └── README.md
```

Ejecución:

Ejecución del código:

```
--- Prueba Ejercicio 1 ---
Nodos esperados: 6
Nodos calculados: 6
```

```
--- Prueba Ejercicio 2 ---
Raíz (Esperado 10): 10
Hijo Izquierdo (Esperado 5): 5
Hijo Derecho (Esperado 15): 15
Hijo Izq del 5 (Esperado 3): 3
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: FEBRERO – JULIO 2026



```
--- Prueba Ejercicio 3 ---
```

```
Altura esperada: 3
```

```
Altura calculada: 3
```

```
Altura de arbol nulo (esperado 0): 0
```

```
--- Prueba Ejercicio 4 ---
```

```
Resultado esperado: [1, 2, 3, 4, 5, 6, 7]
```

```
Tu resultado: [1, 2, 3, 4, 5, 6, 7]
```

```
--- Prueba Ejercicio 5 ---
```

```
Antes de invertir:
```

```
Hijo Izq: 2 | Hijo Der: 3
```

```
Despues de invertir (Esperado: Izq 3 | Der 2):
```

```
Hijo Izq: 3 | Hijo Der: 2
```